

Arkan Home

E-Commerce Data Localization & Business Intelligence Analytics

An end-to-end analytics project transforming raw e-commerce data into a localized Egyptian business intelligence solution using Excel, SQL Server, Star Schema modeling, DAX, and Power BI.

Presented by: Ahmed Magdy · Mark Morris · Mai Ibrahim · Aisha Taha

EXCEL

DATA CLEANING

DATA VALIDATION

SQL SERVER

ETL

STAR SCHEMA

DAX

POWER BI

DASHBOARD DESIGN

BUSINESS INSIGHTS



Project Executive Summary



From raw e-commerce data to localized business intelligence insights

Arkan Home is an end-to-end analytics project that transforms the public Olist e-commerce dataset into a localized Egyptian business case for a simulated Home Decor and Furniture company.

Business Context

Localized the original e-commerce dataset into an Egyptian market scenario focused on Home Decor and Furniture business performance.

Data Engineering

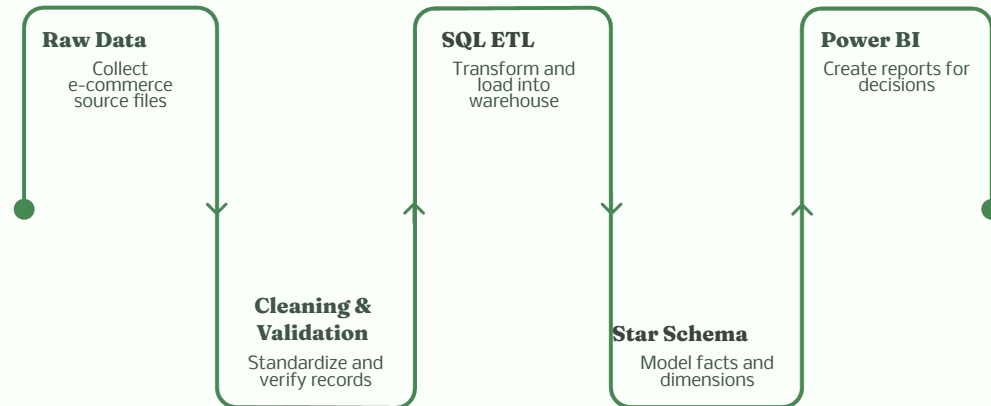
Prepared, cleaned, validated, and transformed raw transactional data using Excel, SQL Server, and structured ETL logic.

Analytical Modeling

Built a Star Schema model with Fact_Sales and dimension tables to support accurate, scalable, and business-ready analytics.

Business Insights

Delivered an interactive Power BI dashboard covering executive performance, sales, logistics, customer behavior, and growth analysis.



Key Message: We did not just build a dashboard. We built a complete business intelligence solution.

Dataset & Localization Strategy



How we transformed the original Olist data into an Egyptian business case

The original Brazilian e-commerce dataset was transformed into a localized Egyptian business scenario to make the analysis more market-relevant and business-oriented.



Source Dataset

Brazilian Olist E-Commerce Dataset. Main source tables: Orders, Order Items, Payments, Reviews, Products, Customers, Sellers.

Localization Assets

- Geography_Mapping.xlsx
- product_category_mapping.csv
- State Flags.csv

Localization Logic

- Mapped original geography to Egyptian cities and regions
- Reclassified product categories into Home Decor & Furniture groups
- Adapted the business scenario to fit a realistic Egyptian e-commerce case



Key Message: We did not just analyze a dataset. We localized it into a market-specific business case.



The SQL layer was built as a controlled data pipeline, starting with raw staging tables, followed by cleaning, validation, ETL transformation, and star schema modeling.

1

Created the initial raw layer inside SQL Server under **Home_Arkan_DB** to receive the original source entities in a structured staging environment.

```

1  ALTER USER hruser IDENTIFIED BY hruser;
2
3  -- Create tables
4
5  CREATE TABLE test_order_headers (
6    order_id VARCHAR(10) NOT NULL,
7    customer_id VARCHAR(10) NOT NULL,
8    order_date DATE NOT NULL,
9    order_status VARCHAR(10) NOT NULL,
10   order_delivered_customer_date DATE NOT NULL,
11   order_delivered_customer_location VARCHAR(10) NOT NULL,
12   order_estimated_delivery_date DATE NOT NULL,
13   CONSTRAINT test_order_headers_pk PRIMARY KEY (order_id)
14 );
15
16 CREATE TABLE test_order_items (
17   order_id VARCHAR(10) NOT NULL,
18   item_id INT NOT NULL,
19   product_id VARCHAR(10) NOT NULL,
20   unit_price DECIMAL(10,2) NOT NULL,
21   quantity INT NOT NULL,
22   CONSTRAINT test_order_items_pk PRIMARY KEY (order_id, item_id)
23 );
24
25 -- Populate
26
27 CREATE TABLE test_order_payments (
28   order_id VARCHAR(10) NOT NULL,
29   payment_id INT NOT NULL,
30   payment_type VARCHAR(10) NOT NULL,
31   payment_installment INT NOT NULL,
32   payment_value DECIMAL(10,2) NOT NULL,
33   CONSTRAINT test_order_payments_pk PRIMARY KEY (order_id, payment_id)
34 );
35
36 -- Randomize
37
38 CREATE TABLE test_order_reviews (
39   review_id VARCHAR(10) NOT NULL,
40   order_id VARCHAR(10) NOT NULL,
41   review_date DATE NOT NULL,
42   review_status VARCHAR(10) NOT NULL,
43   review_customer_title VARCHAR(10) NOT NULL,
44   review_content VARCHAR(100) NOT NULL,
45   review_rating_star DECIMAL(10,2) NOT NULL,
46   review_customer_location VARCHAR(10) NOT NULL,
47   CONSTRAINT test_order_reviews_pk PRIMARY KEY (review_id)
48 );
49
50 -- Products
51
52 CREATE TABLE test_order_products (
53   product_id VARCHAR(10) NOT NULL,

```

2

Validation

```

10 ALTER INDEX idx_products USING BTREE (product_name);
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
```

3

Built mapping tables, dimension tables, and the central **Fact_Sales** table to prepare the data for scalable Power BI reporting.

```

1  -- @author: Nishant Arora
2  -- @Date: 2020-09-25 10:00:00
3  -- @Email: nishant.arora@hike.com
4  -- @Project: Hike
5  -- @Version: 1.0.0
6  -- @License: MIT
7  -- @Copyright: © 2020 Nishant Arora
8  -- @Description: This script is used to create the database schema for the Hike application.
9  -- @Usage: Run the script using the command: mysql -u root -p -h localhost -e $(cat schema.sql)
10 -- @Notes: The script is designed to be run on a MySQL database. It will create the database if it does not exist, and then create the tables and their relationships.
11
12 -- Create database
13 CREATE DATABASE IF NOT EXISTS hike;
14
15 -- Use database
16 USE hike;
17
18 -- Create tables
19
20 -- Create users table
21 CREATE TABLE users (
22     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
23     username VARCHAR(50) NOT NULL,
24     password VARCHAR(50) NOT NULL,
25     email VARCHAR(50) NOT NULL,
26     phone VARCHAR(15) NOT NULL,
27     created_at DATETIME NOT NULL,
28     updated_at DATETIME NOT NULL,
29     INDEX (username),
30     INDEX (password),
31     INDEX (email),
32     INDEX (phone)
33 );
34
35 -- Create products table
36 CREATE TABLE products (
37     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
38     name VARCHAR(50) NOT NULL,
39     description VARCHAR(255) NOT NULL,
40     price DECIMAL(10,2) NOT NULL,
41     category VARCHAR(50) NOT NULL,
42     created_at DATETIME NOT NULL,
43     updated_at DATETIME NOT NULL,
44     INDEX (name),
45     INDEX (description),
46     INDEX (price),
47     INDEX (category)
48 );
49
50 -- Create orders table
51 CREATE TABLE orders (
52     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
53     user_id INT(11) UNSIGNED NOT NULL,
54     product_id INT(11) UNSIGNED NOT NULL,
55     quantity INT(11) UNSIGNED NOT NULL,
56     total_price DECIMAL(10,2) NOT NULL,
57     status VARCHAR(20) NOT NULL,
58     created_at DATETIME NOT NULL,
59     updated_at DATETIME NOT NULL,
60     INDEX (user_id),
61     INDEX (product_id),
62     INDEX (quantity),
63     INDEX (total_price),
64     INDEX (status)
65 );
66
67 -- Create reviews table
68 CREATE TABLE reviews (
69     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
70     user_id INT(11) UNSIGNED NOT NULL,
71     product_id INT(11) UNSIGNED NOT NULL,
72     rating INT(11) UNSIGNED NOT NULL,
73     comment VARCHAR(255) NOT NULL,
74     created_at DATETIME NOT NULL,
75     updated_at DATETIME NOT NULL,
76     INDEX (user_id),
77     INDEX (product_id),
78     INDEX (rating),
79     INDEX (comment)
80 );
81
82 -- Create wishlist table
83 CREATE TABLE wishlist (
84     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
85     user_id INT(11) UNSIGNED NOT NULL,
86     product_id INT(11) UNSIGNED NOT NULL,
87     added_at DATETIME NOT NULL,
88     INDEX (user_id),
89     INDEX (product_id),
90     INDEX (added_at)
91 );
92
93 -- Create favorites table
94 CREATE TABLE favorites (
95     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
96     user_id INT(11) UNSIGNED NOT NULL,
97     product_id INT(11) UNSIGNED NOT NULL,
98     added_at DATETIME NOT NULL,
99     INDEX (user_id),
100    INDEX (product_id),
101    INDEX (added_at)
102 );
103
104 -- Create carts table
105 CREATE TABLE carts (
106     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
107     user_id INT(11) UNSIGNED NOT NULL,
108     product_id INT(11) UNSIGNED NOT NULL,
109     quantity INT(11) UNSIGNED NOT NULL,
110     total_price DECIMAL(10,2) NOT NULL,
111     status VARCHAR(20) NOT NULL,
112     created_at DATETIME NOT NULL,
113     updated_at DATETIME NOT NULL,
114     INDEX (user_id),
115     INDEX (product_id),
116     INDEX (quantity),
117     INDEX (total_price),
118     INDEX (status)
119 );
120
121 -- Create shipping table
122 CREATE TABLE shipping (
123     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
124     user_id INT(11) UNSIGNED NOT NULL,
125     product_id INT(11) UNSIGNED NOT NULL,
126     quantity INT(11) UNSIGNED NOT NULL,
127     total_price DECIMAL(10,2) NOT NULL,
128     status VARCHAR(20) NOT NULL,
129     created_at DATETIME NOT NULL,
130     updated_at DATETIME NOT NULL,
131     INDEX (user_id),
132     INDEX (product_id),
133     INDEX (quantity),
134     INDEX (total_price),
135     INDEX (status)
136 );
137
138 -- Create payments table
139 CREATE TABLE payments (
140     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
141     user_id INT(11) UNSIGNED NOT NULL,
142     product_id INT(11) UNSIGNED NOT NULL,
143     quantity INT(11) UNSIGNED NOT NULL,
144     total_price DECIMAL(10,2) NOT NULL,
145     status VARCHAR(20) NOT NULL,
146     created_at DATETIME NOT NULL,
147     updated_at DATETIME NOT NULL,
148     INDEX (user_id),
149     INDEX (product_id),
150     INDEX (quantity),
151     INDEX (total_price),
152     INDEX (status)
153 );
154
155 -- Create transactions table
156 CREATE TABLE transactions (
157     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
158     user_id INT(11) UNSIGNED NOT NULL,
159     product_id INT(11) UNSIGNED NOT NULL,
160     quantity INT(11) UNSIGNED NOT NULL,
161     total_price DECIMAL(10,2) NOT NULL,
162     status VARCHAR(20) NOT NULL,
163     created_at DATETIME NOT NULL,
164     updated_at DATETIME NOT NULL,
165     INDEX (user_id),
166     INDEX (product_id),
167     INDEX (quantity),
168     INDEX (total_price),
169     INDEX (status)
170 );
171
172 -- Create orders table
173 CREATE TABLE orders (
174     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
175     user_id INT(11) UNSIGNED NOT NULL,
176     product_id INT(11) UNSIGNED NOT NULL,
177     quantity INT(11) UNSIGNED NOT NULL,
178     total_price DECIMAL(10,2) NOT NULL,
179     status VARCHAR(20) NOT NULL,
180     created_at DATETIME NOT NULL,
181     updated_at DATETIME NOT NULL,
182     INDEX (user_id),
183     INDEX (product_id),
184     INDEX (quantity),
185     INDEX (total_price),
186     INDEX (status)
187 );
188
189 -- Create reviews table
190 CREATE TABLE reviews (
191     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
192     user_id INT(11) UNSIGNED NOT NULL,
193     product_id INT(11) UNSIGNED NOT NULL,
194     rating INT(11) UNSIGNED NOT NULL,
195     comment VARCHAR(255) NOT NULL,
196     created_at DATETIME NOT NULL,
197     updated_at DATETIME NOT NULL,
198     INDEX (user_id),
199     INDEX (product_id),
200     INDEX (rating),
201     INDEX (comment)
202 );
203
204 -- Create wishlist table
205 CREATE TABLE wishlist (
206     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
207     user_id INT(11) UNSIGNED NOT NULL,
208     product_id INT(11) UNSIGNED NOT NULL,
209     added_at DATETIME NOT NULL,
210     INDEX (user_id),
211     INDEX (product_id),
212     INDEX (added_at)
213 );
214
215 -- Create favorites table
216 CREATE TABLE favorites (
217     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
218     user_id INT(11) UNSIGNED NOT NULL,
219     product_id INT(11) UNSIGNED NOT NULL,
220     added_at DATETIME NOT NULL,
221     INDEX (user_id),
222     INDEX (product_id),
223     INDEX (added_at)
224 );
225
226 -- Create carts table
227 CREATE TABLE carts (
228     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
229     user_id INT(11) UNSIGNED NOT NULL,
230     product_id INT(11) UNSIGNED NOT NULL,
231     quantity INT(11) UNSIGNED NOT NULL,
232     total_price DECIMAL(10,2) NOT NULL,
233     status VARCHAR(20) NOT NULL,
234     created_at DATETIME NOT NULL,
235     updated_at DATETIME NOT NULL,
236     INDEX (user_id),
237     INDEX (product_id),
238     INDEX (quantity),
239     INDEX (total_price),
240     INDEX (status)
241 );
242
243 -- Create shipping table
244 CREATE TABLE shipping (
245     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
246     user_id INT(11) UNSIGNED NOT NULL,
247     product_id INT(11) UNSIGNED NOT NULL,
248     quantity INT(11) UNSIGNED NOT NULL,
249     total_price DECIMAL(10,2) NOT NULL,
250     status VARCHAR(20) NOT NULL,
251     created_at DATETIME NOT NULL,
252     updated_at DATETIME NOT NULL,
253     INDEX (user_id),
254     INDEX (product_id),
255     INDEX (quantity),
256     INDEX (total_price),
257     INDEX (status)
258 );
259
260 -- Create payments table
261 CREATE TABLE payments (
262     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
263     user_id INT(11) UNSIGNED NOT NULL,
264     product_id INT(11) UNSIGNED NOT NULL,
265     quantity INT(11) UNSIGNED NOT NULL,
266     total_price DECIMAL(10,2) NOT NULL,
267     status VARCHAR(20) NOT NULL,
268     created_at DATETIME NOT NULL,
269     updated_at DATETIME NOT NULL,
270     INDEX (user_id),
271     INDEX (product_id),
272     INDEX (quantity),
273     INDEX (total_price),
274     INDEX (status)
275 );
276
277 -- Create transactions table
278 CREATE TABLE transactions (
279     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
280     user_id INT(11) UNSIGNED NOT NULL,
281     product_id INT(11) UNSIGNED NOT NULL,
282     quantity INT(11) UNSIGNED NOT NULL,
283     total_price DECIMAL(10,2) NOT NULL,
284     status VARCHAR(20) NOT NULL,
285     created_at DATETIME NOT NULL,
286     updated_at DATETIME NOT NULL,
287     INDEX (user_id),
288     INDEX (product_id),
289     INDEX (quantity),
290     INDEX (total_price),
291     INDEX (status)
292 );
293
294 -- Create orders table
295 CREATE TABLE orders (
296     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
297     user_id INT(11) UNSIGNED NOT NULL,
298     product_id INT(11) UNSIGNED NOT NULL,
299     quantity INT(11) UNSIGNED NOT NULL,
300     total_price DECIMAL(10,2) NOT NULL,
301     status VARCHAR(20) NOT NULL,
302     created_at DATETIME NOT NULL,
303     updated_at DATETIME NOT NULL,
304     INDEX (user_id),
305     INDEX (product_id),
306     INDEX (quantity),
307     INDEX (total_price),
308     INDEX (status)
309 );
310
311 -- Create reviews table
312 CREATE TABLE reviews (
313     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
314     user_id INT(11) UNSIGNED NOT NULL,
315     product_id INT(11) UNSIGNED NOT NULL,
316     rating INT(11) UNSIGNED NOT NULL,
317     comment VARCHAR(255) NOT NULL,
318     created_at DATETIME NOT NULL,
319     updated_at DATETIME NOT NULL,
320     INDEX (user_id),
321     INDEX (product_id),
322     INDEX (rating),
323     INDEX (comment)
324 );
325
326 -- Create wishlist table
327 CREATE TABLE wishlist (
328     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
329     user_id INT(11) UNSIGNED NOT NULL,
330     product_id INT(11) UNSIGNED NOT NULL,
331     added_at DATETIME NOT NULL,
332     INDEX (user_id),
333     INDEX (product_id),
334     INDEX (added_at)
335 );
336
337 -- Create favorites table
338 CREATE TABLE favorites (
339     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
340     user_id INT(11) UNSIGNED NOT NULL,
341     product_id INT(11) UNSIGNED NOT NULL,
342     added_at DATETIME NOT NULL,
343     INDEX (user_id),
344     INDEX (product_id),
345     INDEX (added_at)
346 );
347
348 -- Create carts table
349 CREATE TABLE carts (
350     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
351     user_id INT(11) UNSIGNED NOT NULL,
352     product_id INT(11) UNSIGNED NOT NULL,
353     quantity INT(11) UNSIGNED NOT NULL,
354     total_price DECIMAL(10,2) NOT NULL,
355     status VARCHAR(20) NOT NULL,
356     created_at DATETIME NOT NULL,
357     updated_at DATETIME NOT NULL,
358     INDEX (user_id),
359     INDEX (product_id),
360     INDEX (quantity),
361     INDEX (total_price),
362     INDEX (status)
363 );
364
365 -- Create shipping table
366 CREATE TABLE shipping (
367     id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
368     user_id INT(11) UNSIGNED NOT NULL,
369     product_id
```

 Key Message: The SQL pipeline transformed raw data into a trusted analytical foundation. This layer prepared the project for accurate modeling, DAX measures, and Power BI reporting.

Power BI Dashboard Architecture



Connecting model design, navigation, and interactive business reporting

The Power BI solution was designed as a connected analytical experience, combining a branded landing page, structured data model, interactive navigation, slicers, and dashboard pages.



Dashboard Ecosystem

- Landing Page
- Executive Overview
- Sales
- Logistics
- Customer Behavior
- Growth Analysis

Model Foundation

- Central Fact_Sales table
- Connected dimension tables
- Consistent relationship logic
- Clean filtering and scalable reporting

Interactivity & UX

- Page navigation
- Year and month slicers
- Cross-filtering across visuals
- Help overlay and guided user flow



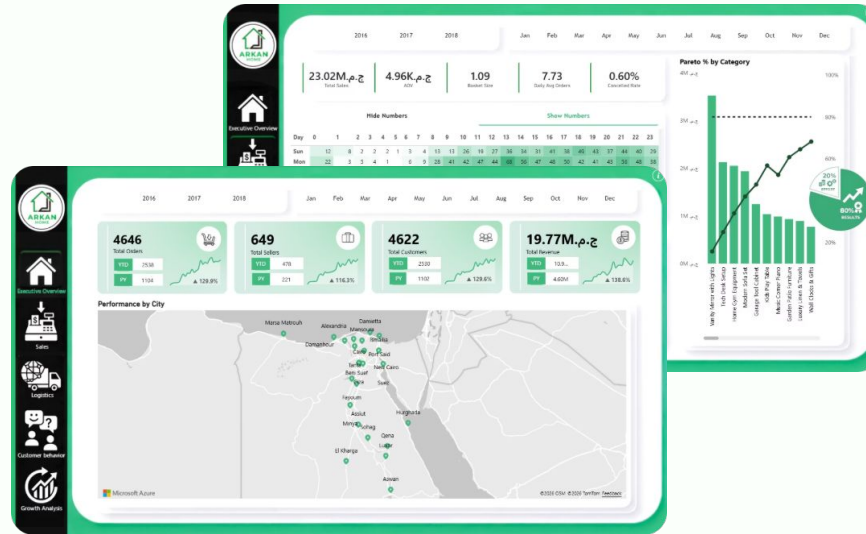
Key Message: The Power BI layer converted the analytical model into a guided business intelligence experience. It connected navigation, visuals, and business domains into one decision-support system.

Executive & Sales Insights



Combining leadership overview with commercial performance analysis

The executive and sales dashboards provide a clear view of business scale, revenue performance, order behavior, category contribution, payment methods, and market distribution.



Executive Overview

- Total Orders: **4,646**
- Total Sellers: **649**
- Total Customers: **4,622**
- Total Revenue: **19.77M**

Sales Performance

- Total Sales: **23.02M**
- AOV: **4.96K**
- Basket Size: **1.09**
- Daily Avg Orders: **7.73**
- Cancelled Rate: **0.60%**

Commercial Analysis

- Hourly order heatmap
- AOV trend by year and month
- Sales by payment method
- Pareto analysis by category



Key Message: Executive and sales analytics translate transactions into commercial intelligence. They help stakeholders understand performance, revenue drivers, and market activity.

Logistics & Customer Behavior Insights



Connecting operational performance with customer experience

The logistics and customer behavior dashboards help connect fulfillment performance, delivery reliability, acquisition patterns, retention signals, review scores, and customer satisfaction.



Logistics Performance

- Avg Dispatch Days: **3.17**
- Late Dispatch %: **8.99%**
- Avg Delivery Days: **12.16**
- Late Delivery %: **8.12%**
- Late Deliveries: **368**

Customer Behavior

- Total Customers: **4,622**
- Returning Customers: **24**
- Orders per Customer: **1.01**
- Avg Customer Spend: **4.98K**
- Avg CSAT Score: **4.08**

Decision Support Value

- Identifies logistics bottlenecks
- Highlights customer satisfaction signals
- Connects delivery performance with customer experience
- Supports service improvement actions



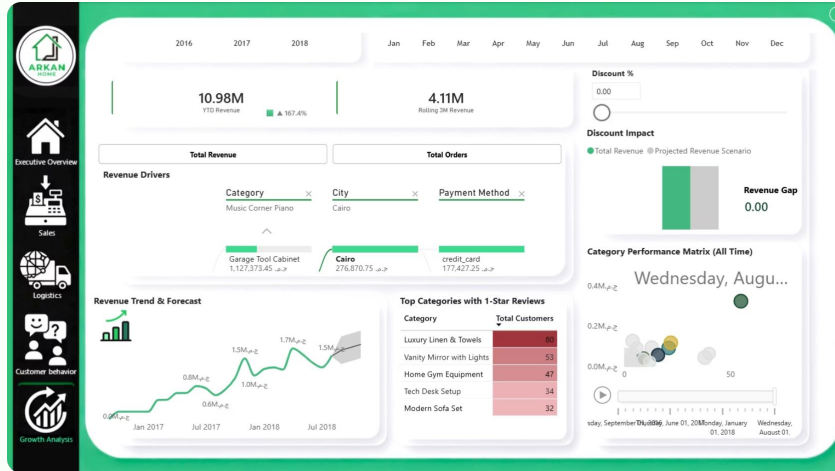
Key Message: Operational intelligence becomes stronger when logistics and customer behavior are analyzed together. This view connects delivery performance with satisfaction, retention, and service improvement opportunities.

Growth Analysis & Decision Support



Turning business performance into strategic planning insights

The growth analysis dashboard supports strategic review by combining revenue trends, category contribution, discount scenario analysis, forecast movement, and performance risk indicators.



Growth Drivers

- Revenue driver decomposition
- Category, city, and payment method impact
- Top categories with weak review signals

Scenario Thinking

- Discount impact simulation
- Projected revenue comparison
- Revenue gap visibility
- Revenue trend and forecast review

Decision Support

- Supports growth planning
- Identifies revenue leakage areas
- Reveals category-level risks
- Connects performance with action priorities



Key Message: Growth analysis turns performance history into future-focused decision support. It helps compare revenue drivers, scenarios, and category risks in one analytical view.

Technical Skills, DAX & Challenges



The technical foundation behind the final business intelligence solution

The project demonstrates a full BI workflow across data review, SQL engineering, analytical modeling, DAX logic, dashboard design, and business analysis.

Skills Applied

Excel & Data Review

SQL Server

Data Cleaning

ETL

Star Schema

DAX

Power BI

Business Analysis

DAX & KPI Logic

- Total Revenue
- AOV
- Basket Size
- Daily Avg Orders
- Late Delivery %
- Returning Customers
- Avg CSAT Score
- Projected Revenue Scenario

Challenges Solved

- Data quality inconsistencies
- Localization of a non-Egyptian dataset
- Modeling raw transactions into star schema
- Designing a usable multi-page dashboard experience



Key Message: The project proves capability across the full BI lifecycle. Technical execution and business analysis were combined into one complete decision-support solution.

Final Outcomes & Closing



From raw data to localized business intelligence

Arkan Home transformed raw e-commerce data into a complete localized BI solution that supports executive review, sales analysis, logistics monitoring, customer understanding, growth planning, and decision support.

End-to-End BI Solution

From raw data preparation to final dashboard storytelling.

Localized Business Case

Converted the original dataset into an Egyptian Home Decor and Furniture scenario.

Decision-Support Dashboard

Delivered interactive pages for executive, sales, logistics, customer, and growth analysis.

Team: Ahmed Magdy · Mark Morris · Mai Ibrahim · Aisha Taha

Thank You



From raw data to business intelligence. A complete analytics journey designed to support smarter decisions.